

Sesiunea 2: CSS modern și selectoare DOM

Capitolul 1: Fundamentele CSS și Box Model - De la Schelet HTML la Prezentare Vizuală	2
Obiective de învățare	2
1.1 Ce este CSS-ul și de ce contează pentru QA?	2
1.2 Metode de aplicare a CSS-ului (Separation of Concerns)	3
1.3 Anatomia unei reguli CSS	4
1.4 Modelul Cutiei (The Box Model) - Concept Vital pentru QA	4
1.5 Știați că...?	5
1.6 Poveste Aplicată: „Designerul și Zona de Confort”	5
1.7 Exerciții Practice	6
1.8 Răspunsuri la Întrebări & Soluții la Exerciții	6
1.9 🤖 Mentalitatea de Inginer în Era AI: Locul tău pe piață	6
Capitolul 2: Selectoare CSS de bază - Instrumentele de precizie ale QA-ului	8
Obiective de învățare	8
2.1 Sintaxa de bază: Element, ID și Clasă	8
2.2 Selectoarele de Atribute: "Sfințitul Graal" al Automatizării	9
2.3 Poveste Aplicată: "Livrarea Coletului"	9
2.4 Exerciții Practice	10
2.5 Răspunsuri la Întrebări & Soluții la Exerciții	10
2.6 🤖 Mentalitatea de Inginer în Era AI: Locul tău pe piață	10
Capitolul 3: Selectoare CSS Avansate & Pseudo-clase - Navigarea complexă în DOM	11
Obiective de învățare	11
3.1 Navigarea prin Arbore: Selectoare Relationale (Combinators)	12
3.2 Pseudo-clase de Stare: Testarea Dinamismului	13
3.3 Pseudo-clase Structurale: Găsirea „Acului în Carul cu Fân”	13
3.4 Poveste Aplicată: "Adresa Completă"	14
3.5 Exerciții Practice	14
3.6 Răspunsuri la Întrebări & Soluții la Exerciții	15

3.7 🤖 Mentalitatea de Inginer în Era AI: Locul tău pe piață	15
Capitolul 4: Layout Modern (Flexbox de bază) & Tema Sesiunii 2	16
Obiective de învățare	16
4.1 De ce trebuie un QA să înțeleagă Flexbox?	17
4.2 Flexbox: Conceptele de bază (Cea mai scurtă explicație)	17
4.3 Poveste Aplicată: "Raftul din Supermarket"	18
4.4 Exerciții Practice	18
4.5 Răspunsuri la Întrebări & Soluții la Exerciții	18
4.6 Tema pentru Acasă (Proiect Sesiunea 2)	19
4.7 🤖 Mentalitatea de Inginer în Era AI: Locul tău pe piață	19
Capitolul 5: Bonus & Extra Practice - Nivel Avansat (Pentru viitorii Seniori)	21
5.1 Provocarea 1: Coșmarul Claselor Dinamice (Tailwind CSS)	21
5.2 Provocarea 2: Inamicul Invizibil (Z-Index & Suprapuneri)	22
5.3 🗝️ Cheile de Rezolvare (Soluții & Explicații)	23

Capitolul 1: Fundamentele CSS și Box Model - De la Schelet HTML la Prezentare Vizuală

Obiective de învățare

🧠 La finalul acestui capitol, vei fi capabil să:

- Explici rolul CSS-ului în ecosistemul web și separarea conceptelor (Separation of Concerns).
- Conectezi corect regulile de stil la un document HTML folosind metoda recomandată în industrie.
- Descompui anatomia unei reguli CSS și să identifice erorile de sintaxă.
- Înțelegi "Modelul Cutiei" (Box Model) pentru a investiga de ce elementele se suprapun sau blochează interacțiunile.
- Corelezi proprietățile CSS de vizibilitate cu potențialele erori din testele E2E (End-to-End).

1.1 Ce este CSS-ul și de ce contează pentru QA?

Dacă în Sesiunea 1 am stabilit că HTML-ul reprezintă "cărămizile și structura" casei, CSS-ul (Cascading Style Sheets) reprezintă vopseaua, tapetul, mobila și dimensiunile exacte ale încăperilor. HTML definește *ce* este un element (un buton, un paragraf), iar CSS definește *cum arată și unde este poziționat* acel element pe ecran.

Ca viitor QA Automation Engineer, te-ai putea întreba: *"De ce trebuie să învăț design web dacă eu voi scrie teste de funcționalitate?"*

Răspunsul stă în provocările zilnice ale automatizării:

- **Localizarea Elementelor:** Playwright și Cypress folosesc nativ "Selectoare CSS" pentru a găsi elemente în pagină. Fără a stăpâni CSS, nu vei ști cum să-i indici robotului cu precizie chirurgicală unde să acționeze.
- **Stările de Vizibilitate:** Foarte multe bug-uri și teste picate apar pentru că un element este prezent în DOM (deci HTML-ul este corect), dar o regulă CSS îl face invizibil (display: none, visibility: hidden sau opacity: 0). Playwright, fiind construit să emuleze un om, va refuza să dea click pe un element ascuns.
- **Interceptarea Click-urilor:** Un alt element invizibil poate fi randat *peste* butonul tău (ex: un overlay de încărcare). Cunoașterea CSS te ajută să depistezi "inamicul" în DevTools.

1.2 Metode de aplicare a CSS-ului (Separation of Concerns)

Există trei moduri de a colora un element HTML, dar industria folosește predominant doar unul. Este important să le recunoști pe toate la inspecția codului:

- **Inline CSS (Direct pe element):** Se aplică folosind atributul style direct pe tag-ul HTML. Este greu de menținut și evitat în aplicațiile moderne.
`<button style="color: red;">Click</button>`
- **Internal CSS (În <head>):** Se scrie între etichetele <style> în interiorul documentului HTML. E util doar pentru pagini foarte simple.
- **External CSS (Fișier extern - Standardul Industriei):** Păstrează codul curat. HTML-ul stă într-un fișier, designul în altul (ex: style.css). Aceasta respectă principiul de *Separation of Concerns*.

Pentru metoda externă (pe care o vom folosi la aplicația "Task Tracker"), facem legătura folosind tag-ul <link> în secțiunea <head>:

```
<head>
  <title>Task Tracker</title>
```

```
<!-- Legătura critică între structură și design -->
<link rel="stylesheet" href="style.css">
</head>
```

1.3 Anatomia unei reguli CSS

Codul CSS este compus dintr-o listă de reguli. Fiecare regulă îi spune browserului cum să deseneze unul sau mai multe elemente. Iată cum arată o regulă completă și corectă:

```
button {
  background-color: #3498db;
  color: white;
  border-radius: 5px;
}
```

Să o descompunem:

- **Selectorul (button):** Indică ținta din documentul HTML. (Îi spunem browserului: "Găsește absolut toate butoanele de pe pagină!").
- **Blocul de declarații ({ ... }):** Acoladele încadrează pachetul de reguli vizuale care se vor aplica țintei.
- **Proprietatea (background-color, color):** Ce caracteristică specifică vrem să schimbăm (ex: culoarea de fundal, culoarea textului).
- **Valoarea (#3498db, white):** Cum vrem să setăm acea proprietate.
- **Atenție maximă:** Fiecare declarație (pereche proprietate-valoare) se desparte prin două puncte (:) și se termină obligatoriu cu punct și virgulă (;)! Omiterea celui ; va face ca regula următoare să nu funcționeze.

1.4 Modelul Cutiei (The Box Model) - Concept Vital pentru QA

Unul dintre cele mai importante secrete pe care trebuie să le știi este că, pentru browser, **ABSOLUT FIECARE ELEMENT HTML ESTE O CUTIE DREPTUNGHILARĂ**. Chiar dacă un buton arată rotund (border-radius), "amprenta" lui fizică pe ecran este un dreptunghi.

Acest dreptunghi este definit de **Box Model**, care are 4 straturi (dinspre interior spre exterior):

- **Content (Conținutul):** Inima cutiei. Textul sau imaginea propriu-zisă.
- **Padding (Umplutura):** Spațiul *interior*, dintre text și marginea cutiei. Face butonul să pară

mai "gras" și mai ușor de apăsat. Dacă dai click pe padding, dai click pe buton.

- **Border (Chenarul):** Linia care delimitează marginea elementului. Poate fi invizibilă, continuă sau punctată.
- **Margin (Marginea exterioară):** Spațiul *exterior*, gol, dintre acest element și elementele vecine. Marginea împinge alte elemente la distanță. **Dacă un QA încearcă să dea click pe "Margin", click-ul trece prin el și lovește elementul din spate!**

1.5 Știați că...?

💡 Știați că...

De ce se numește "Cascading" (în cascadă)? Dacă scrii două reguli CSS care se bat cap în cap pentru același element (ex: sus scrii că butonul e roșu, mai jos scrii că e verde), browserul va aplica regula citită ultima. Designul "curge în cascadă" de sus în jos. Singura excepție este adăugarea flag-ului !important la finalul unei reguli, care forțează suprascrierea cascadei.

Proprietatea pointer-events: none; este inamicul invizibil al QA-ului. Dezvoltatorii o folosesc frecvent pentru a dezactiva temporar un element (un efect de "disabled"). Butonul arată complet normal pe ecran, dar dacă încerci să dai click pe el (manual sau din codul Playwright), click-ul trece pur și simplu prin el. Fără CSS, ai raporta că Playwright e stricat!

1.6 Poveste Aplicată: „Designerul și Zona de Confort”

Gândește-te la un Tablou pe care vrei să-l agăți pe perete.

HTML-ul este pânza însăși (Content).

Designerul CSS vine și spune:

- "Pune-i un Passepartout alb, lat de 5 centimetri, între pânză și ramă" (Acesta este **Padding-ul**).
- "Pune-i o ramă groasă de lemn de stejar" (Acesta este **Border-ul**).
- "Nu agăța niciun alt tablou la o distanță mai mică de 20 de centimetri de acesta. Are nevoie de spațiu să respire!" (Acesta este **Margin-ul**).

Când tu, ca Automation Tester, programezi robotul Playwright să dea click pe tablou, robotul va da click fix în mijlocul Content-ului. Dar dacă Margin-ul unui alt tablou uriaș se suprapune

accidental peste tabloul tău (din cauza unui CSS prost scris de developeri), robotul va lovi Margin-ul transparent al celui alt element și va eșua! Așa se nasc 30% din testele E2E eșuate în viața reală.

1.7 Exerciții Practice

Exercițiul 1: Investighează CSS-ul în DevTools

- Deschide Google Chrome și intră pe www.wikipedia.org.
- Dă click-dreapta pe butonul central de căutare (cel cu o lupă) și alege **Inspect**.
- În panoul DevTools, sub tab-ul "Elements", vei vedea secțiunea **Styles**.
- **Misiunea 1:** Găsește proprietatea background-color (sau color) și schimb-o din paleta de culori. Observă modificarea live.

Exercițiul 2: Explorează Box Model-ul (Investigație Avansată)

- Tot în DevTools, lângă tab-ul "Styles", caută tab-ul **Computed** (sau derulează până la capătul panoului Styles dacă ești pe un ecran mic).
- Vei vedea o diagramă cu niște cutii colorate una într-alta (albastru, verde, galben, portocaliu). Acesta este Box Model-ul redat grafic de Chrome!
- **Misiunea 2:** Trece cu mouse-ul peste dreptunghiul verde (padding) și peste cel portocaliu (margin). Observă cum pe ecranul propriu-zis (pe site), Chrome va colora cu verde spațiul interior al elementului și cu portocaliu spațiul exterior. Câți pixeli are padding-ul superior al acelu buton?

1.8 Răspunsuri la Întrebări & Soluții la Exerciții

Soluție Exercițiul 1 & 2:

Dacă ai urmat pașii, ai reușit să vizualizezi atât regulile de stil brute, cât și **Box Model-ul** calculat de browser!

Panoul **Computed** este instrumentul de top al unui QA pentru a rezolva erorile de tipul *"Playwright dă click pe elementul greșit"*. Când treci cu mouse-ul peste margin/padding în acea diagramă, Chrome evidențiază direct pe pagină zonele "moarte" sau spațiile invizibile care împing elementele unele în altele. Ai făcut un pas uriaș către depanarea profesionistă!

1.9 🤖 Mentalitatea de Inginer în Era AI: Locul tău pe piață

🔊 QA vs. AI: De ce cunoașterea CSS-ului te face de neînlocuit...

Odată cu ascensiunea asistenților bazați pe LLM (ChatGPT, GitHub Copilot, Cursor), mulți începători se întreabă: "Dacă AI-ul îmi poate scrie CSS-ul sau îmi poate genera direct selectorul pentru testul meu Playwright, eu ce rol mai am?"

Ce face AI-ul bine (și cum îl folosești): AI-ul este un generator excelent de *sintaxă*. Îi poți cere *"generează-mi o clasă CSS pentru un buton albastru rotund"* sau *"scrie comanda Playwright pentru a da click pe buton"*. El va executa rapid: `page.click('button')`.

Capcana (Unde AI-ul eșuează): AI-ul este orb la *contextul vizual*. În producție, butonul acela ar putea fi acoperit de un **Margin** transparent al unui element vecin (așa cum am învățat la secțiunea Box Model). Când rulezi testul generat de AI, acesta va pica cu o eroare de tipul: *"Element is intercepted"*. Dacă îi dai eroarea înapoi AI-ului, el va sugera soluții superficiale, de tip "bandaj" (ex: folosește parametrul `{ force: true }`), care forțează click-ul ocolind regulile UI-ului. Făcând asta, AI-ul tocmai te-a ajutat să ascunzi un bug real pe care un utilizator uman l-ar experimenta!

Cum gândește un inginer: Un inginer știe că `{ force: true }` este un compromis periculos. El va deschide **DevTools**, se va uita în panoul **Computed** și va investiga straturile cutiei (Box Model). Inginerul va trage concluzia de arhitectură: *"Problema nu e la test, problema e că header-ul are un padding prea mare care se suprapune peste zona de conținut"*.

De ce vei rămâne relevant pe piață: În era AI, rolul tău evoluează de la *"cel care scrie cod de la zero"* la **Orchestrator, Investigator și Arhitect**. Roboții pot scrie cod, dar nu pot depana vizual aplicații și nu înțeleg *intenția* din spatele designului. Cunoștințele tale fundamentale (cum funcționează de fapt CSS și HTML "sub capotă") devin **super-puterea ta de debugging**. Tu ești cel care ghidează AI-ul, îi validează rezultatele și previne introducerea de "soluții oarbe" în sistem.

Capitolul 2: Selectoare CSS de bază - Instrumentele de precizie ale QA-ului

Obiective de învățare



La finalul acestui capitol, vei fi capabil să:

- Identifici și scrii selectoare CSS de bază: Element, ID, Clasă.
- Înțelegi specificitatea selectoarelor și de ce ID-ul este regele.
- Scrii selectoare bazate pe attribute (ex: data-testid), fundamentale în testarea E2E.
- Evitați "capcanele" claselor dinamice generate de framework-uri moderne (React, Angular).

2.1 Sintaxa de bază: Element, ID și Clasă

Așa cum un poștaș are nevoie de o adresă pentru a livra o scrisoare, CSS-ul are nevoie de **Selectoare** pentru a ști cărui element îi aplică regulile. Mai târziu, vei folosi *aceeași adresă* pentru a-i spune lui Playwright unde să dea click!

Există trei metode fundamentale de selecție:

1. Selectorul de Element (Type Selector): Cel mai slab (Genericul)

- **Cum arată:** Folosește direct numele tag-ului HTML.
- **Exemplu CSS:** `button { background-color: blue; }`
- **În Playwright:** `page.locator('button')`
- **Problema:** Va ținti *absolut toate* butoanele de pe pagină. Dacă ai un buton de "Login" și unul de "Cancel", acest selector le va lovi pe amândouă, iar testul tău va eșua din cauză că e prea ambiguu (eroare: strict mode violation).

2. Selectorul de Clasă (Class Selector): Moderatul (Grupul)

- **Cum arată în HTML:** `<div class="card-produs">...</div>`
- **Sintaxa CSS:** Începe întotdeauna cu un punct (.) urmat de numele clasei.
- **Exemplu CSS:** `.card-produs { border: 1px solid black; }`
- **În Playwright:** `page.locator('.card-produs')`

- **Cum îl folosim:** O clasă poate fi aplicată pe zeci de elemente diferite. Ca QA, folosești selectorul de clasă când vrei să numeri câte produse sunt într-un coș, nu când vrei să dai click pe un anumit produs.

3. Selectorul de ID (ID Selector): Puternicul (Unicul)

- **Cum arată în HTML:** `<input id="email-field">`
- **Sintaxa CSS:** Începe întotdeauna cu un diez/hashtag (#).
- **Exemplu CSS:** `#email-field { width: 100%; }`
- **În Playwright:** `page.locator('#email-field')`
- **De ce este preferat:** Conform regulilor HTML, ID-ul trebuie să fie unic pe toată pagina. Dacă îi spui robotului "Găsește #email-field", el nu va avea nicio ezitare.

2.2 Selectoarele de Atribute: "Sfințitul Graal" al Automatizării

În aplicațiile moderne complexe (create cu React, Vue sau Angular), clasele și id-urile se schimbă adesea dinamic sau automat (ex: o clasă poate arăta așa: `<button class="btn_v2_xyz789">`). Dacă scrii un test bazat pe o astfel de clasă, testul va pica mâine, când codul va deveni `btn_v2_abc123`.

Cum rezolvă QA-ul această problemă de instabilitate? Folosind **Selectoare de Atribute!**

- **Cum arată în HTML:** `<button data-testid="submit-login">Log In</button>`
- **Sintaxa CSS:** Se scriu între paranteze drepte [].
- **Exemplu CSS:** `[data-testid="submit-login"] { color: white; }`
- **În Playwright:** `page.locator('[data-testid="submit-login"]')` sau comanda specială și rapidă `page.getByTestId('submit-login')`.

Această metodă decuplează total designul (clasele/id-urile care se schimbă des) de logică testării!

2.3 Poveste Aplicată: "Livrarea Coletului"

Gândește-te la selectoare ca la moduri de a găsi un om într-un bloc:

- **Selectorul de Element (button):** "Caută un Om!" (Sunt mulți oameni în bloc, nu știi la care să lași pachetul).
- **Selectorul de Clasă (.baiat-cu-par-blond):** "Caută băieții cu părul blond!" (S-ar putea să găsești 3 pe aceeași scară, e încă neclar).
- **Selectorul de ID (#cnp-1900101123456):** "Caută omul cu acest CNP" (Perfect, l-ai găsit, e unic. Dar ce te faci dacă și-a schimbat adresa și nu ți-a zis?).

- **Selectorul de Atribut ([data-rol="administrator-bloc"]):** "Dă coletul oricui are ecusonul oficial de administrator". (Asta e abordarea supremă în QA. Chiar dacă îți schimbă freza (clasa) sau buletinul (ID-ul), ecusonul rămâne stabil!).

2.4 Exerciții Practice

Exercițiul 1: Vânătoarea de Selectoare (DevTools)

- Mergi pe google.com (sau orice motor de căutare preferi).
- Inspectează butonul "Caută pe Google" (sau echivalentul).
- În panoul HTML (Elements), uită-te atent la eticheta <input> sau <button>.
- **Misiunea 1:** Care este o clasă CSS aplicată pe acest buton? Ce ID are (dacă are)? Are vreun atribut special, cum ar fi data-ved sau aria-label?

Exercițiul 2: Scrie propriul Selector de Atribut

În aplicația "Task Tracker", developerul a adăugat următorul cod:

```
<div id="container-99" class="wrapper-blue" data-test="user-profile-card">Ion Popescu</div>
```

Cum ai scrie un selector CSS valid (folosind parantezele drepte) care să se lege strict de atributul de test, ignorând ID-ul și clasa? Notează-l.

2.5 Răspunsuri la Întrebări & Soluții la Exerciții

Soluție Exercițiul 2:

Selectorul CSS corect (cel mai sigur pentru viitorul tău test automatizat) este:

```
[data-test="user-profile-card"]
```

Bravo! Dacă știi să scrii asta, știi să scrii nucleul a 80% din acțiunile Playwright.

2.6 🧠 Mentalitatea de Inginer în Era AI: Locul tău pe piață

QA vs. AI: Capcana Generatoarelor de Cod ...

Ce face AI-ul bine (și cum îl folosești): Unelte ca GitHub Copilot sau funcția de înregistrare (Codegen) din Playwright sunt geniale la a genera rapid zeci de linii de cod de test. Apeși "Record", dai click pe un buton de login, iar AI-ul generează: `page.locator('.bg-blue-500.hover:bg-blue-700.text-white.font-bold').click()`. A funcționat repede!

Capcana (Unde AI-ul eșuează): AI-ul (sau generatorul) este adesea leneș sau orb la intenție. Selectorul de mai sus, bazat pe clase utilitare (ex: Tailwind CSS), este un "selector fragil" (brittle selector). Dacă mâine designerul decide ca butonul de login să fie puțin mai deschis la culoare (bg-blue-400), testul tău va eșua catastrofal, deși logica de business e intactă. O companie nu își permite să repare 500 de teste doar pentru că s-a schimbat o nuanță de albastru.

Cum gândește un inginer: Inginerul QA știe că testele trebuie să fie independente de design. El va șterge codul fragil generat de AI și va cere dezvoltatorilor: "Avem nevoie de un data-testid='login-button' aici". Apoi va rescrie el însuși linia: `page.getByTestId('login-button').click()`.

De ce vei rămâne relevant pe piață: AI-ul poate scrie cod rapid, dar tu ești Arhitectul Stabilității. Rolul tău nu este doar să creezi scripturi, ci să construiești un sistem de testare robust, imun la schimbările vizuale minore. **Valoarea ta stă în deciziile de strategie (ce selectoare folosim la nivel de echipă), nu în simpla generare de text.**

Capitolul 3: Selectoare CSS Avansate & Pseudo-clase - Navigarea complexă în DOM

Obiective de învățare

🧠 La finalul acestui capitol, vei fi capabil să:

- Folosești selectoarele de relație (Copil, Descendent, Frate) pentru a naviga în structuri HTML complexe.
- Înțelegi și aplici pseudo-clasele de stare (ex: :hover, :disabled) pentru a testa interacțiunile utilizatorului.
- Utilizezi pseudo-clasele structurale (ex: :nth-child) pentru a găsi elemente specifice în liste sau tabele, esențial când ID-urile lipsesc.
- Înlanțui corect multiple selectoare pentru a obține o precizie maximă în localizarea elementelor țintă (Chaining).

3.1 Navigarea prin Arbore: Selectoare Relationale (Combinators)

Până acum, am ținut elementele direct, ca și cum am avea adresa lor exactă (ID sau Clasă). Dar ce te faci când ai o listă cu 10 produse, toate cu exact aceeași clasă .produs, fără niciun ID, și vrei să dai click doar pe produsul din secțiunea "Recomandări"?

Aici intervin **Combinatorii**. Ei ne permit să descriem drumul prin DOM (ierarhia Părinte-Copil-Frate învățată în Sesiunea 1).

1. Selectorul Descendent (Spațiul Gol): "Căutare în Adâncime"

- **Sintaxă:** ElementA ElementB (separat prin spațiu).
- **Cum funcționează:** Găsește toate ElementB care se află *oriunde în interiorul* lui ElementA (fie că sunt copii direcți, nepoți sau strănepoți).
- **Exemplu:** #meniu a { ... } - Va găsi toate link-urile (<a>) din interiorul elementului cu ID-ul meniu, ignorând link-urile din subsolul paginii.

2. Selectorul Copil Direct (>): "Căutare Strictă"

- **Sintaxă:** ElementA > ElementB.
- **Cum funcționează:** Găsește ElementB DOAR dacă este copilul imediat, de gradul 1, al lui ElementA. Ignoră nepoții.
- **Exemplu**

HTML:

```
<ul class="lista-principala">
```

```
<li>Item 1</li>
<li>
  <ul class="sublista">
    <li>Sub-Item 1</li> <!-- Acesta e un nepot pentru .lista-
principala -->
  </ul>
</li>
</ul>
```

- **Exemplu CSS:** `.lista-principala > li { ... }` - Va selecta doar "Item 1" și elementul `<li>` care conține sublista. Nu va selecta "Sub-Item 1", pentru că acela nu e copil direct al clasei `.lista-principala`.

3. Înlanțuirea (Chaining): "Fără Spațiu!"

- **Sintaxă:** `ElementA.ClasaB#IdC` (fără niciun spațiu între ele).
- **Cum funcționează:** Combină condițiile *pe același element*. "Găsește un buton CARE ARE Și clasa de alertă Și atributul data-test".
- **Exemplu:** `button.btn-danger[data-status="error"]` - Extrem de util în automatizare pentru a filtra elementele cu precizie chirurgicală.

3.2 Pseudo-clase de Stare: Testarea Dinamismului

Paginile web nu sunt poze; ele reacționează la utilizator. Un buton arată într-un fel când pagina se încarcă, altfel când treci cu mouse-ul peste el și altfel când este dezactivat. CSS-ul gestionează aceste stări prin **Pseudo-clase** (recunoscute prin semnul două puncte : la început).

- **:hover:** Când cursorul mouse-ului este deasupra elementului.
 - *Focus QA:* Verificăm adesea dacă un meniu ascuns devine vizibil doar la `:hover`. (În Playwright vom folosi comanda `page.locator(...).hover()`).
- **:focus:** Când elementul (de obicei un `<input>`) este activ, fiind selectat cu tasta Tab sau dând click în el pentru a scrie.
- **:disabled:** Găsește elementele din formulare care sunt blocate (ex: butonul de "Submit" înainte de a bifa "Sunt de acord cu termenii").
 - *Exemplu Playwright:* `expect(page.locator('button[type="submit"]')).toBeDisabled();`
(Acest assertion se bazează direct pe starea DOM reflectată de pseudo-clasa `:disabled`).
- **:checked:** Găsește checkbox-urile sau butoanele radio care sunt bifate.

3.3 Pseudo-clase Structurale: Găsirea „Acului în Carul cu Fân”

Aceasta este tehnica de "luptă" a QA-ului. Ești în fața unui tabel cu 50 de rânduri. Toate sunt identice. Dezvoltatorul a plecat în concediu și a uitat să pună attribute data-testid pe ele. Cum dai click pe rândul al 3-lea?

- **:first-child / :last-child:** Selectează primul sau ultimul element dintr-un grup de frați.
 - *Exemplu:* ul.meniu > li:last-child (Țintește mereu ultimul link din meniu, indiferent câte adaugă programatorul în viitor).
- **:nth-child(n):** Aici e magie! Poți specifica exact un număr (index).
 - *Exemplu:* tr:nth-child(3) (Găsește al 3-lea rând dintr-un tabel).
 - *Truc Avansat:* Poți folosi odd (impar) sau even (par) pentru a ținti rânduri alternative. tr:nth-child(even) { background-color: grey; }.

3.4 Poveste Aplicată: "Adresa Completă"

Imaginează-ți că trebuie să livrezi un pachet într-un zgârie-nori.

- **Selectorul Simplu (.usa):** Cauți orice ușă din clădire. Misiune imposibilă.
- **Selectorul Descendent (#etaj-5 .usa):** "Du-te la etajul 5, caută prin toate birourile, dulapurile și camerele tehnice și găsește orice are eticheta de 'usa'". (Tot e o arie mare de căutare).
- **Selectorul Copil Direct (#hol-principal > .usa):** "Du-te pe holul principal și intră DOAR pe prima ușă pe care o vezi lipită de acest hol. Nu intra mai adânc în camere!". (Eficiență maximă).
- **Pseudo-clasa Structurală (#hol-principal > .usa:nth-child(3)):** "Du-te pe holul principal, numără ușile, deschide-o pe a 3-a". Asta face un QA când nu are ID-uri!
- **Înlănțuirea (#hol-principal > .usa.rosie:disabled):** "A 3-a ușă era stricată? Bine. Du-te pe holul principal, găsește ușa care e roșie ȘI pe care e pus lacătul (dezactivată)".

3.5 Exerciții Practice

Exercițiul 1: Tabelul Problematic

Deschide aplicația noastră "Task Tracker" din Sesiunea 1 (fișierul index.html în browser).

Ai acolo un tabel de "Istoric Task-uri". Imaginează-ți că tabelul are 5 rânduri <tr> și vrei să scrii un selector CSS care să țintească *strict* al doilea rând din tabel (cel cu id-ul #1002 dacă l-ai adăugat în exercițiul anterior).

Cum ai scrie acest selector folosind selectorul descendent și pseudo-clasa :nth-child? Notează

pe o foaie!

Exercițiul 2: Navigarea în DevTools (Investigație)

- Mergi pe www.wikipedia.org.
- Inspectează lista de limbi (cea din jurul globului central).
- În panoul Elements, apasă Ctrl+F (sau Cmd+F pe Mac). Va apărea o bară de căutare în partea de jos a codului HTML. Aici poți scrie selectoare CSS pentru a vedea dacă găsești elemente!
- **Misiunea ta:** Scrie selectorul `div.central-featured-lang > strong` în acea bară de căutare. Câte rezultate primești? Ce s-a întâmplat dacă înlocuiești semnul `>` cu un spațiu simplu (selectorul descendent)?

3.6 Răspunsuri la Întrebări & Soluții la Exerciții

Soluție Exercițiul 1:

Cel mai sigur și precis selector pentru al doilea rând din corpul tabelului ar fi:

```
#history-table tbody tr:nth-child(2)
```

Am combinat ID-ul tabelului (pentru a izola căutarea doar în zona acelui tabel), descendentul `tbody` (pentru a evita antetul `thead`) și `tr:nth-child(2)` pentru rândul dorit. Acesta este nivelul de precizie așteptat de la un QA Automation!

3.7 🧠 Mentalitatea de Inginer în Era AI: Locul tău pe piață

🔍 QA vs. AI: Navigarea în Ierarhiile Complexe (DOM Traversal) ...

Ce face AI-ul bine (și cum îl folosești): Dacă îi dai AI-ului o structură HTML cu un tabel și îi ceri: "Vreau un selector Playwright care să îmi dea textul din coloana 3, rândul 5", unelte ca ChatGPT sau Copilot îți vor genera aproape instant un șir CSS sau XPath lung, uneori perfect funcțional pe moment, de exemplu: `page.locator('div > table > tbody > tr:nth-child(5) > td:nth-child(3)').textContent()`. Este un asistent excelent pentru o scurtătură rapidă de memorie.

Capcana (Unde AI-ul eșuează): AI-ul este de cele mai multe ori fixat pe structura curentă pe care o "vede". Selectorul generat mai sus (`div > table > ...`) este extrem de fragil. Un astfel de

selector înălțuit orbește este o bombă cu ceas. Dacă mâine developerul adaugă un mic container div de design (`<div class="wrapper">`) chiar înainte de tabel, structura părinte-copil se modifică. Testul creat de AI va eșua pe producție cu "Timeout - Element nu a fost găsit".

Cum gândește un inginer: Un QA Automation Engineer gândește în termeni de "reziliență". El se uită la selectorul AI-ului și spune: "Nu mă voi baza pe 5 nivele de părinți și copii, e prea periculos." El va restrânge căutarea folosind un element de ancoră (un ID apropiat) și abia apoi va folosi un selector de descendent. Va optimiza sugestia AI-ului în: `page.locator('#date-table-id tr:nth-child(5) td:nth-child(3)').textContent()`. A eliminat pașii inutili, iar testul a devenit rezistent la adăugarea de noi div-uri de design (deoarece spațiul gol permite căutarea la orice adâncime).

De ce vei rămâne relevant pe piață: În timp ce LLM-urile sunt bune la a genera "răspunsuri la primul impuls" pe baza unor modele statistice de sintaxă, ele nu pot evalua riscul schimbărilor viitoare ale bazei de cod a companiei. Ca inginer, nu ești angajat să scrii sintaxă (AI-ul face asta), ești angajat să prevezi ce se va rupe și să construiești teste care supraviețuiesc refactorizărilor de cod. **Mentenanța reprezintă 80% din costul automatizării; valoarea ta este reducerea acestui cost prin decizii arhitecturale inteligente.**

Capitolul 4: Layout Modern (Flexbox de bază) & Tema Sesiunii 2

Obiective de învățare

🧠 La finalul acestui capitol, vei fi capabil să:

- Înțelegi conceptul de Flexbox și relația Părinte-Copil (Container-Items) la nivel vizual.
- Corelezi comportamentul elementelor flexibile cu rularea testelor E2E pe diferite rezoluții (Mobile vs. Desktop).
- Folosești badge-ul flex din DevTools pentru a depana suprapunerile de elemente.

- Realizezi Tema Sesiunii 2, aplicând clase, ID-uri și structuri flexibile, gata pentru validarea în platformă.

4.1 De ce trebuie un QA să înțeleagă Flexbox?

În trecut, așezarea elementelor pe o pagină (layout-ul) era un coșmar: se foloseau tabele ascunse sau calcule matematice complicate cu pixeli. Astăzi, 99% din aplicațiile moderne folosesc **Flexbox** (Flexible Box Layout) sau CSS Grid.

Ca QA Automation Engineer, te vei lovi de Flexbox într-un scenariu clasic: **Testarea pe rezoluții multiple (Responsiveness)**.

Playwright permite rularea aceluiași test pe o rezoluție de desktop (ex: 1920x1080) și pe o rezoluție de iPhone (ex: 375x667). Când ecranul se micșorează, un container *Flexbox* va muta butoanele, le va ascunde într-un "hamburger menu" sau le va pune unele sub altele. Dacă nu înțelegi cum funcționează acest comportament elastic, nu vei ști de ce testul tău trece pe desktop, dar crapă pe mobil cu eroarea *"Element is not visible"*.

4.2 Flexbox: Conceptele de bază (Cea mai scurtă explicație)

Sistemul Flexbox se bazează strict pe relația Părinte-Copil învățată în Capitolul 3.

1. Containerul Flex (Părintele):

Pentru a activa magia, trebuie să adăugăm proprietatea `display: flex;` pe elementul părinte.

```
.menu-navigare {  
  display: flex;  
}
```

Din acel moment, toți copiii direcți ai acestui meniu devin "Flex Items" și se vor așeza implicit pe un singur rând (ca niște mărgelile pe o ață).

2. Alinierea pe axa principală (`justify-content`):

Această proprietate controlează cum sunt distribuite elementele pe rând:

- `flex-start`: Toate elementele se adună la stânga.

- center: Toate elementele stau la mijloc.
- space-between: Primul element se lipește de stânga, ultimul de dreapta, iar restul au spațiu egal între ele (extrem de folosit pentru bara de navigație: Logo la stânga, Buton Login la dreapta).

3. Trecerea pe un rând nou (flex-wrap):

Dacă ai 10 butoane pe un ecran mic de telefon, ele vor ieși în afara ecranului. Proprietatea flex-wrap: wrap; le permite să cadă pe rândul următor când nu mai au spațiu, prevenind tăierea lor vizuală (și salvându-ți testele automatizate!).

4.3 Poveste Aplicată: "Raftul din Supermarket"

Imaginează-ți un raft lung într-un supermarket (Acesta este **Părintele / Flex Container**).

Pe raft pui 5 cutii de cereale (Acestea sunt **Copiii / Flex Items**).

- Dacă managerul magazinului zice: *"Împingeți toate cutiile în stânga"* -> Asta face justify-content: flex-start;
- Dacă managerul zice: *"Puneți o cutie la un capăt, una la celălalt, și lăsați loc egal între restul"* -> Asta face justify-content: space-between;
- Dacă vine un client și împinge raftul făcându-l mai scurt (Simularea vizionării de pe telefon), cutiile s-ar strivi. Dar dacă managerul a pus regula flex-wrap: wrap;; cutia care nu mai încapă pe raft "cade" automat pe raftul de mai jos, la vedere. Exact așa face un UI bun pentru a preveni bug-urile vizuale!

4.4 Exerciții Practice

Exercițiul 1: Badge-ul Flex din DevTools

Chrome DevTools are un instrument grafic genial pentru Flexbox, ascuns la vedere.

- Deschide Google Chrome și intră pe www.github.com.
- Dă click-dreapta pe meniul de sus (bara de navigație cu logo, search, sign in) și dă **Inspect**.
- În panoul Elements, uită-te după tag-urile HTML (<header>, <div>, <nav>). Lângă unele dintre ele vei vedea un mic buton gri (badge) pe care scrie flex.
- **Misiunea ta:** Dă click pe acel badge flex. Ce se întâmplă pe ecran?

4.5 Răspunsuri la Întrebări & Soluții la Exerciții

Soluție Exercițiul 1:

Când dai click pe badge-ul flex din DevTools, Chrome va "desena" pe ecran (printr-o grilă hașurată) exact structura invizibilă a containerului flexibil. Îți va arăta spațiile goale (space-between) și marginile exacte ale copiilor. Acesta este cel mai rapid mod vizual de a verifica de ce un buton nu stă acolo unde ar trebui sau de ce acoperă un alt element!

4.6 Tema pentru Acasă (Proiect Sesiunea 2)

Sarcina ta (Task-ul de business):

Trebuie să transformi scheletul HTML simplu al "Task Tracker-ului" din Sesiunea 1 într-o aplicație structurată, pregătită atât vizual (cu un layout flexibil de bază), cât și tehnic (cu clase, id-uri și data-testid-uri) pentru prima noastră interacțiune cu Playwright.

Cerințe Tehnice (Acceptance Criteria pentru validarea platformei):

- **Fișier CSS Extern:** Documentul tău HTML trebuie să aibă un `<link>` către un fișier CSS (chiar dacă momentan trimiți doar HTML-ul în platformă, tag-ul de link trebuie să existe pentru arhitectură, având `href="style.css"`).
- **Meniu Flexibil (Navbar):**
 - În `<header>`, adaugă un `<nav>` care să aibă clasa `navbar`.
 - În interiorul acestui `<nav>`, trebuie să existe un element cu id-ul `logo` și un buton de login având atributul `data-testid="btn-login"`.
- **Selector structural (Tabelul):**
 - Ai tabelul cu istoricul task-urilor. Asigură-te că al 3-lea rând din `<tbody>` (adică elementul `<tr>`) conține un buton de "Șterge" având clasa `delete-row`. (Acest lucru va dovedi că poți folosi pseudo-clasa `:nth-child(3)` mai târziu în Playwright).
- **Buton cu Stare Dinamică:**
 - În formularul de adăugare task, adaugă un `<button>` de "Submit" și pune-i nativ atributul HTML `disabled` (adică butonul să fie dezactivat implicit, simulând că formularul e gol). Trebuie să aibă `data-testid="submit-task-btn"`.

4.7 Mentalitatea de Inginer în Era AI: Locul tău pe piață

🔊 QA vs. AI: Depanarea Anomaliilor de Layout (Responsiveness) ...

Ce face AI-ul bine (și cum îl folosești): Dacă îi ceri unui AI (Claude, ChatGPT): "Scrie-mi un cod CSS Flexbox pentru un meniu de navigare care pune logo-ul la stânga și meniul la dreapta", îți va genera în 2 secunde un cod perfect valid cu `display: flex; justify-content: space-between;`. E un asistent grozav pentru a scrie CSS boilerplate.

Capcana (Unde AI-ul eșuează): Când rulezi suita de teste Playwright pe CI/CD (GitHub Actions), s-ar putea ca unul dintre teste să ruleze într-un viewport îngust (simulare iPad Mini). AI-ul a scris meniul, dar a uitat să testeze ce se întâmplă când textul e prea lung: elementele Flexbox se vor strivi unele în altele, iar butonul tău de "Login" va fi acoperit de un alt strat. Testul va pica subit. Dacă îi dai eroarea AI-ului, el s-ar putea să sugereze un selector mai complicat sau folosirea lui `{ force: true }`, ocolind exact problema reală.

Cum gândește un inginer: Un QA Automation Engineer nu ascunde mizeria sub preș. El rulează testul în modul UI sau folosește Playwright Trace Viewer, observă ecranul micșorat și își dă seama de comportamentul Flexbox-ului. Inginerul deschide un bug pe frontend: "Pe rezoluții sub 768px, Flex container-ul din header nu face wrap, cauzând interceptarea butonului de Login de către zona de Logo".

De ce vei rămâne relevant pe piață: AI-ul analizează cod static, dar aplicațiile sunt medii dinamice și fluide. Nu ești plătit să știi sintaxa Flexbox pe de rost, ești plătit să înțelegi cum fizica din spatele layout-ului web impactează acțiunile utilizatorului (și, implicit, ale scripturilor tale). **Abilitatea ta de a lega un test E2E picat de un comportament eronat de CSS Flexbox te face un inginer veritabil, capabil să investigheze, nu doar un simplu rulator de scripturi.**

Capitolul 5: Bonus & Extra Practice - Nivel Avansat (Pentru viitorii Seniori)

Acest capitol este 100% opțional. Dacă ai înțeles conceptele de bază din capitolele 1-4, ești deja pregătit pentru Playwright. Dar dacă ești genul de inginer căruia îi place să desfacă lucrurile pentru a vedea cum funcționează și vrei să fii pregătit pentru cele mai dificile interviuri tehnice, ai ajuns în locul potrivit!

Aici nu mai lucrăm în medii ideale (unde dezvoltatorii ne pun frumos data-testid peste tot). Aici lucrăm în "Vestul Sălbatic" al codului legacy.

5.1 Provocarea 1: Coșmarul Claselor Dinamice (Tailwind CSS)

Scenariul:

Ești angajat la un startup care folosește un framework CSS modern (precum **Tailwind** sau **Styled Components**). Clasele HTML arată ca o supă de litere și se schimbă la fiecare lansare de cod. Nu ai voie să modifice codul sursă pentru a adăuga un ID.

Ai următorul cod HTML pentru un coș de cumpărături:

```
<div class="flex-col w-full px-4 py-2 mt-8 wrapper-xyz">
  <div class="item-row flex justify-between border-b pb-2">
    <span class="text-sm font-bold">Abonament Premium</span>
    <button class="bg-red-500 hover:bg-red-700 text-white rounded px-2"
aria-label="Sterge Abonament">X</button>
  </div>
  <div class="item-row flex justify-between border-b pb-2 mt-4">
    <span class="text-sm font-bold">Taxă Procesare</span>
    <button class="bg-gray-300 text-gray-500 rounded px-2"
disabled>X</button>
  </div>

  <div class="checkout-area mt-10">
    <button class="btn-primary flex items-center justify-center w-full
py-4 rounded-lg bg-green-500 text-white font-bold" aria-label="Finalizare
Comanda">
      <span>Mergi la Plată</span>
      <svg class="icon-arrow">...</svg>
    </button>
```

```
</div>
</div>
```

Misiunea 1.A: Scrie un selector CSS ultra-robust care să găsească butonul de "Mergi la Plată". Nu ai voie să folosești *nicio clasă* CSS (.btn-primary, .bg-green-500 etc.) pentru că mâine se pot schimba.

Misiunea 1.B: Scrie un singur selector CSS care să țintească *doar* butoanele de ștergere ("X") care sunt **active** (adică să îl ignori pe cel dezactivat de la "Taxă Procesare"). Hint: Folosește pseudo-clasa de negație :not().

5.2 Provocarea 2: Inamicul Invizibil (Z-Index & Suprapuneri)

Scenariul:

Rulezi testul Playwright care face click pe butonul de "Salvează". Testul pică cu eroarea:

```
Error: locator.click: Target closed. Element <button id="save-
btn">Salvează</button> is intercepted by <div class="loading-
overlay"></div>.
```

Te duci în browser să verifici manual. Butonul "Salvează" se vede perfect clar pe ecran. Apeși pe el și funcționează. Ești confuz. De ce zice robotul că e interceptat dacă ție îți merge?

Misiunea 2:

Deschizi codul și observi asta:

```
<button id="save-btn">Salvează Datele</button>
<div class="loading-overlay"></div>
```

Și următorul CSS scris de un coleg Junior:

```
.loading-overlay {
  position: absolute;
  top: 0;
  left: 0;
  width: 100vw;
  height: 100vh;
```

```
opacity: 0; /* A făcut overlay-ul invizibil! */
z-index: 9999;
}
```

Cum îi explici developerului de ce a picat testul tău (ce a greșit el din punct de vedere al Box Model-ului / Vizibilității)? Și ce **proprietate CSS** magică ar trebui să adauge pe .loading-overlay pentru ca "click-ul" tău să treacă efectiv prin acel overlay transparent și să lovească butonul?

5.3 🐞 Cheile de Rezolvare (Soluții & Explicații)

Nu trișa! Citește asta doar după ce ai încercat să rezolvi singur provocările.

Soluție Provocarea 1 (Coșmarul Claselor):

- **1.A (Butonul Mergi la Plată):** Când clasele sunt volatile, ne bazăm pe attribute de accesibilitate (aria-*). Acestea sunt stabile pentru că sunt folosite de cititoarele de ecran pentru nevăzători.

Selectorul corect:

```
button[aria-label="Finalizare Comanda"]
```

Explicație: Acest selector este imun la redesign. Indiferent cât se schimbă Tailwind-ul, funcția butonului rămâne aceeași.

- **1.B (Negația):** Combinăm atributul de accesibilitate cu pseudo-clasa :not().

Selectorul corect:

```
button[aria-label="Sterge Abonament"]:not(:disabled)
```

Explicație: Îi spunem robotului "Găsește butoanele care au label-ul de ștergere, DAR exclude-le pe cele care au starea :disabled". Acesta este un selector de nivel de Senior!

Soluție Provocarea 2 (Inamicul Invizibil):

- **Explicația pentru Developer:** "Salut! Ai setat opacity: 0 pe overlay. Asta îl face 100% transparent pentru ochiul uman, dar **fizic, cutia lui (Box Model) este încă desenată pe ecran** deasupra tuturor elementelor (pentru că are z-index: 9999). Când eu (sau Playwright) dau click, noi de fapt lovim acest div transparent, nu butonul de dedesubt. Faptul că a mers manual a fost o coincidență (probabil overlay-ul a dispărut din DOM imediat după ce ai

terminat tu de mișcat mouse-ul)."

- **Soluția CSS:** Proprietatea magică ce rezolvă asta este `pointer-events: none;`. Dacă developerul adaugă `pointer-events: none;` la `.loading-overlay`, browserul va ști să ignore complet acel strat la orice interacțiune cu mouse-ul, permițând click-ului să "cadă" direct pe butonul `#save-btn` de dedesubt!

🎉 **Felicitări!** Dacă ai înțeles aceste două concepte (`aria-labels` și `pointer-events`), ai depășit deja 80% din problemele cu care se confruntă testerii de automatizare în primele lor 6 luni de carieră!